

Aim:

You are a DevOps engineer at a data center in Pune, managing a critical web server that supports millions of users across India. One day, the server crashes, leaving behind a massive, unsorted log file of user requests with Unix timestamps. To help the DevOps team trace the exact sequence of events leading to the failure, you must sort the log entries chronologically.

This is essentially a sorting problem, where timestamps act as keys. Efficient algorithms like Merge Sort or Quick Sort are used to handle large datasets.

Algorithm Design

The program implements Quick Sort to arrange Unix timestamps in ascending order. Here's the structured design:

1. Input Phase

- Read number of log entries n .
- Read n Unix timestamps into an array.

2. Quick Sort Phase

- **Partition:** Choose the last element as pivot.
- Rearrange elements so that those $\leq$ pivot are on the left, and those $>$ pivot are on the right.
- Return the partition index.
- **Recursive Sort:** Apply Quick Sort to left and right subarrays.

3. Output Phase

- Print the sorted timestamps in chronological order.

Algorithm Analysis**1. Time Complexity**

- Best Case: $O(n \log n)$ when partitions are balanced.
- Average Case: $O(n \log n)$.
- Worst Case: $O(n^2)$ when the array is already sorted or reverse sorted (pivot choice is poor).

2. Space Complexity

- $O(\log n)$ due to recursive stack calls.
- In-place sorting, so no extra arrays are needed.

3. Suitability

- Efficient for large datasets (millions of log entries).
- Performs well in practice compared to simpler algorithms like Bubble Sort or Insertion Sort.
- Merge Sort guarantees $O(n \log n)$ but requires extra memory; Quick Sort is memory-efficient.

Input Format:

- The first line contains an integer n , representing the number of log entries.
- The second line contains n Unix timestamps separated by spaces.

Output Format:

- Display the Unix timestamps in chronological order (ascending order), one timestamp per line.

Note:

- Partial code is provided. Complete the missing logic in **swap()**, **partition()**, and **quickSort()** to implement the Quick Sort algorithm.

Source Code:

```
#include <stdio.h>

void swap(long long* a, long long* b) {
    long long temp = *a;
    *a = *b ;
    *b = temp;

    // swap two elements
}

int partition(long long arr[], int low, int high) {
    long long pivot = arr[high];
    int i = low -1;
    for(int j=low;j<high;j++){
        if(arr[j]<=pivot){
            i++;
            swap(&arr[i],&arr[j]);
        }
    }
    swap(&arr[i+1],&arr[high]);
    return i+1;
    // place pivot in correct position and rearrange array
}

void quickSort(long long arr[], int low, int high) {

    // perform partition if segment is valid
    if(low<high){
        int pi = partition(arr,low,high);

        // recursively sort left and right partitions
        quickSort(    arr,low,pi-1    );

        quickSort(    arr,pi+1,high    );
    }
}

int main() {
    int n;
    printf("Log entries: ");
    scanf("%d", &n);
    if (n <= 0) return 0;
    long long logs[n];
    printf("Timestamps:\n");
    for (int i = 0; i < n; i++) {
```

```

        scanf("%lld", &logs[i]);
    }
    quickSort(logs, 0, n - 1);
    printf("Sorted Timestamps:\n");
    for (int i = 0; i < n; i++) {
        printf("%lld\n", logs[i]);
    }
    return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Log entries:
1
Timestamps:
1717305600
Sorted Timestamps:
1717305600

Test Case - 2
User Output
Log entries:
4
Timestamps:
1717305600 1717312800 1717320000 1717327200
Sorted Timestamps:
1717305600
1717312800
1717320000
1717327200